PROGRAMA DE DESENVOLVIMENTO DE COMPETÊNCIAS EM SISTEMAS DIGITAIS

MÓDULO 3 – PROJETO FINAL

ENTREGA 1 – PLANEJAMENTO, MICROARQUITETURA E MODELO DE REFERÊNCIA

CAROLINA CORREIA, GUSTAVO SANTIAGO, LEANDRO GAMA E VALÉRIO SILVA

1. Repositório

Repositório GitHub `github.com/Leandro-Gama/riscv-dma-matrix-coprocessor`. Primeiro commit `25b2935` (30/10/2026, "Estrutura inicial"). Branches: `main` recebe as entregas e `develop` o trabalho corrente.

Diretório	Conteúdo
`01.Specs/`	enunciado e divisão de tarefas
`02.Architecture/`	cronograma, decisões de arquitetura (ADR), contrato de interfaces, diagramas
`03.Reference/`	modelo de referência (golden model) em Python
`04.RTL/`	RTL SystemVerilog: `common`, `soc`, `cpu`, `dma`, `accel`
`05.Verification/`	testbenches unitários, modelos, ambientes de subsistema e de sistema (UVM)
`06.Firmware/`	firmware RISC-V (fonte e `.hex`)
`07.Synthesis/`	scripts, restrições e relatórios do Genus
`08.Docs/`	especificação, relatórios e slides

`make check` roda os testes Python, o lint Verible e a elaboração no Xcelium, sem precisar de licença. A especificação completa da entrega está em `08.Docs/e1_especificacao.md`.

2. Diagrama de blocos

A CPU RV32I configura e dispara a operação; o DMA move os dados e coordena o acelerador; o acelerador é uma malha weight-stationary de `ARRAY_SIZE` × `ARRAY_SIZE` (2, 4 ou 8) com elementos de `DATA_WIDTH` bits (8, 16 ou 32). Todos os acessos de dados passam por uma interconexão AXI4-Lite de 32 bits (ADR 0001).

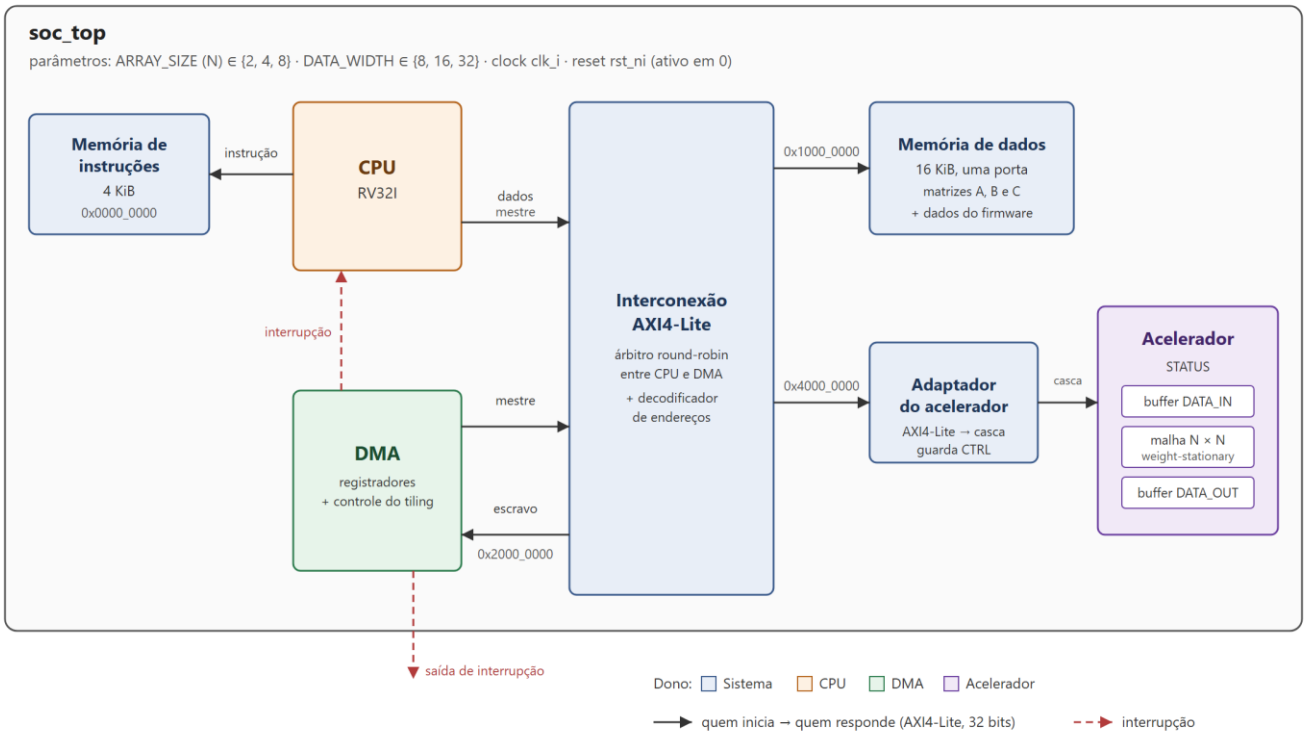


Figura 1 – Diagrama de blocos do soc_top.

Bloco	Dono	Papel
CPU	CPU	executa o firmware; mestre AXI4-Lite pela porta de dados
Memória de instruções	Sistema	4 KiB; só a porta de instrução da CPU, fora do barramento
Interconexão AXI4-Lite	Sistema	2 mestres (CPU, DMA) × 3 escravos; árbitro round-robin + decodificador
Memória de dados	Sistema	16 KiB, uma porta; A, B, C e dados do firmware
DMA	DMA	escravo (registradores) e mestre (move os dados e comanda o acelerador)
Adaptador do acelerador	Sistema	AXI4-Lite → sinais da casca; guarda CTRL e gera os pulsos
Acelerador	Acelerador	buffers DATA_IN/DATA_OUT, malha e máquina de estados

3. Mapa de memória

Base	Região	Tamanho	Acesso
`0x0000_0000`	memória de instruções	4 KiB	porta de instrução da CPU
`0x1000_0000`	memória de dados	16 KiB	CPU e DMA (AXI4-Lite)
`0x2000_0000`	registradores do DMA	4 KiB	CPU (AXI4-Lite)
`0x4000_0000`	acelerador	64 KiB	DMA e CPU (AXI4-Lite)

Endereço sem escravo recebe DECERR. Regiões padrão das matrizes (o firmware pode usar outras, via registradores do DMA):

Endereço	Conteúdo	Tamanho
`0x1000_0000`	dados, constantes e pilha do firmware	4 KiB
`0x1000_1000`	matriz A (linha a linha, `DATA_WIDTH/8` bytes por elemento)	4 KiB
`0x1000_2000`	matriz B (linha a linha, `DATA_WIDTH/8` bytes por elemento)	4 KiB
`0x1000_3000`	matriz C (linha a linha, 32 bits por elemento)	4 KiB

Registradores do DMA (base `0x2000_0000`; todos zeram no reset):

Offset	Nome	Acesso	Conteúdo
`0x00`	`ADDR_A`	RW	endereço inicial de A
`0x04`	`ADDR_B`	RW	endereço inicial de B
`0x08`	`ADDR_C`	RW	endereço inicial de C
`0x0C`	`DIM_M`	RW	linhas de A e de C
`0x10`	`DIM_K`	RW	colunas de A = linhas de B
`0x14`	`DIM_N`	RW	colunas de B e de C
`0x18`	`CONTROL`	RW	bit 0 `START` (escrever 1 dispara); bit 1 `IRQ_EN`
`0x1C`	`STATUS`	RO	bit 0 `BUSY`; bit 1 `DONE`; bit 2 `ERROR`

- `START` com `BUSY` = 1 é ignorado; `ADDR_*`/`DIM_*` com `BUSY` = 1 recebem SLVERR.
- `DONE` sobe após a resposta da última escrita de C; erro de barramento, erro do acelerador ou dimensão 0 terminam com `DONE` e `ERROR`.
- `irq_o` = `DONE` e `IRQ_EN`.

Registradores do acelerador (base `0x4000_0000`, definidos pelo dono do acelerador):

Offset	Nome	Acesso	Conteúdo
`0x0000`	`CTRL`	RW	bit 0 `MODE` (0 pesos, 1 cálculo); bits 1–3 `LOAD`, `START`, `CLR_ERROR` (pulsos)
`0x0004`	`STATUS`	RO	bits 1:0 `STATE` (parado, carregando, calculando); bit 2 `DONE`; bit 3 `ERROR`
`0x1000`	`DATA_IN`	WO	buffer de entrada, `ARRAY_SIZE`² elementos
`0x2000`	`DATA_OUT`	RO	buffer de saída, `ARRAY_SIZE`² elementos

4. Sinais de interface

Barramento AXI4-Lite (endereço e dados de 32 bits; `awprot`/`arprot` em 0):

Canal	Sinais	Sentido
endereço de escrita	`awaddr`, `awvalid`, `awready`	mestre → escravo
dados de escrita	`wdata`, `wstrb`, `wvalid`, `wready`	mestre → escravo
resposta de escrita	`bresp`, `bvalid`, `bready`	escravo → mestre
endereço de leitura	`araddr`, `arvalid`, `arready`	mestre → escravo
dados de leitura	`rdata`, `rresp`, `rvalid`, `rready`	escravo → mestre

- Transferência no ciclo com VALID e READY em 1; VALID não espera READY e fica estável até o aceite.
- Uma transação por vez por mestre; respostas OKAY, SLVERR e DECERR; bytes little-endian na faixa `addr[1:0]`.
- Clock único `clk_i`; reset `rst_ni` assíncrono, ativo em 0.

Portas de cada bloco (direções vistas pelo bloco):

Bloco	Porta	Função
CPU	`imem_addr_o`, `imem_rdata_i` (32)	busca de instrução, leitura no mesmo ciclo
CPU	`m_axi_*`	porta de dados, mestre AXI4-Lite (byte e meia palavra via `wstrb`)
CPU	`irq_i`	interrupção de fim do DMA (uso opcional)
DMA	`s_axi_*`	escravo AXI4-Lite: registradores
DMA	`m_axi_*`	mestre AXI4-Lite: lê A e B, grava C, comanda o acelerador
DMA	`irq_o`	interrupção de fim
Acelerador	`mode_i`, `load_i`, `start_i`, `clr_error_i`	MODE e pulsos de um ciclo vindos de CTRL
Acelerador	`status_o` (32)	conteúdo de STATUS
Acelerador	`data_req_i`, `data_we_i`, `data_addr_i` (16)	pedido de acesso a DATA_IN/DATA_OUT (proposta)
Acelerador	`wdata_i`, `rdata_o` (32), `ready_o`, `error_o`	dado e fim do acesso ao buffer

Coordenação DMA ↔ acelerador pedida no enunciado:

Enunciado	No sistema
requisição	escrita em `CTRL.LOAD` ou `CTRL.START`
pronto/ocupado	`STATUS.STATE`
conclusão	`STATUS.DONE`

5. Modelo de referência (golden model)

`03.Reference/golden/`, Python só com a biblioteca padrão (roda no Python 3.9 da WSL). Calcula C = A × B direto e por blocos, com recorte dos blocos de borda; tamanho e ordem dos blocos, largura dos operandos (8, 16, 32, com ou sem sinal) e do acumulador são parâmetros. A ordem padrão `nkm` mantém um bloco de B fixo enquanto os blocos de A passam (weight-stationary).

Módulo	Função
`numeric.py`	`NumericFormat`: faixas, checagem dos operandos, estouro do acumulador em complemento de 2
`matrix.py`	`generate`: matrizes aleatórias, zeros, máximos, mínimos e identidade, por seed
`tiling.py`	`TileConfig` e `tile_schedule`: sequência de blocos em qualquer das 6 ordens
`matmul.py`	`matmul_reference` (direta) e `matmul_tiled` (por blocos, somando cada parcial em C)

Testes (`make test`, Python 3.9.25 na WSL):

```
test_tiled_matches_reference (tests.test_golden.MatmulTest) ... ok
test_accumulator_wraps (tests.test_golden.MatmulTest) ... ok
test_extreme_values (tests.test_golden.MatmulTest) ... ok
test_every_product_is_covered_once (tests.test_golden.TileScheduleTest) ... ok
test_order_matches_independent_oracle (tests.test_golden.TileScheduleTest) ... ok
test_against_numpy (tests.test_golden.MatmulTest) ... skipped 'numpy not available'
Ran 16 tests in 0.702s
OK (skipped=1)
```

No Windows, com numpy: `python -m pytest 03.Reference -q` → `16 passed, 897 subtests passed`. Execução nas duas matrizes de teste (por blocos × direta, 8 bits com sinal, seed 1):

```
16x16x16 N=2 ordem=nkm blocos= 512 C[0][0]=   2126 OK
16x16x16 N=4 ordem=nkm blocos=  64 C[0][0]=   2126 OK
16x16x16 N=8 ordem=nkm blocos=   8 C[0][0]=   2126 OK
17x13x11 N=2 ordem=nkm blocos= 378 C[0][0]=   6177 OK
17x13x11 N=4 ordem=nkm blocos=  60 C[0][0]=   6177 OK
17x13x11 N=8 ordem=nkm blocos=  12 C[0][0]=   6177 OK
```

6. Decisões pendentes

Com quem	Decisão	Prazo
Acelerador	confirmar `data_req_i`, `data_we_i` e a largura de `data_addr_i`	07/11
Acelerador	formato dos números: `DATA_WIDTH` e sinal	03/11
Acelerador	ordem dos elementos de cada bloco em `DATA_IN` e `DATA_OUT`	07/11
Acelerador	largura dos resultados em `DATA_OUT` (proposta: 32 bits)	07/11
Acelerador	escrita de vários elementos numa palavra de 32 bits	07/11
DMA e acelerador	onde somar os resultados parciais ao longo de K e em que ordem percorrer os blocos	07/11
CPU	core parando até a resposta AXI4-Lite; memórias fora do core	07/11 (portas), 24/11 (core)
Sistema	síntese do sistema com memórias de profundidade parametrizada	antes da síntese final